

ASTRA MOVE

Sistema de Controle e Monitoramento Operacional da Base
Lunar
Turma: 1ECA
Temas: Base e Residência & Mobilidade

Anderson Luiz Sanches Junior — RM: 571834

Vitor Andrade de Oliveira — RM: 570907

João Pedro Moraes Basques — RM: 571710

Gabriel de Oliveira — RM: 569455

Yagu naoto Iwanaga RM: 570825

1. Objetivo:

O nosso projeto, Astra Move, consiste no desenvolvimento de um sistema em python voltado ao controle operacional de uma base lunar, cobrindo a gestão de habitats e a mobilidade da frota. O objetivo principal é garantir a segurança da tripulação e a eficiência logística através de dois programas. Um módulo autônomo de diagnóstico ambiental e de frota, e um simulador interativo para planejamento de transporte e resgates de emergência.

2. Dados selecionados

Para a estruturação da solução, o sistema consome dados dos kits fornecidos utilizando diferentes estruturas de dados. Listas e dicionários são empregados na organização do inventário de rovers, módulos residenciais, ordens de carga e rotas de deslocamento. Matrizes são utilizadas para o mapeamento do terreno e nos acessos logísticos da base. As tuplas armazenam os limites de segurança, como limites de CO₂, O₂ e bateria mínima, mantendo a integridade dos parâmetros.

3. Indicadores Operacionais:

O processamento das informações gera indicadores essenciais para o monitoramento da base. No suporte à vida, o programa calcula a taxa de ocupação total dos habitats e monitora os níveis críticos de CO₂ (acima de 1000 ppm) e O₂ (abaixo de 20,5%) por módulo. Na logística de mobilidade, calcula a autonomia média da frota, a distância total percorrida em cada trajeto considerando ida e volta, e a estimativa do tempo de deslocamento em horas.

4. Decisão Esperada:

A matriz de decisão do simulador avalia as restrições físicas de capacidade de carga e autonomia energética em relação à rota escolhida. O sistema retorna a decisão de “missão autorizada” quando o veículo cumpre os requisitos de carga e bateria em trajetos de baixo risco. Retorna “missão autorizada com restrições” quando a operação é viável, mas a rota exige atenção por apresentar risco médio ou alto. Por fim, determina “missão negada” sempre que houver insuficiência de autonomia do rover ou excesso de carga útil.

Programa 1:

```

programa1.py
1 from mobilidadekit_dados import veiculos, leituras_operacionais, mapa_terreno
2 from residenciaikit_dados import modulos, leituras_ambientais, mapa_base
3
4 limites_seguranca = (1000.0, 20.5, 80.0)
5
6 ARTE_FINAL = r"""
7 |   |   |   |   |   |   | \ /    |   | / \    *
8 |   |   |   |   |   |   | \ /    |   | / \
9 * |   |   |   |   |   |   | - - - |   | - - - *
10 |   |   |   |   |   |   | - - - |   | - - -
11 |   |   |   |   |   |   | - - - |   | - - -
12 |   |   |   |   |   |   | - - - |   | - - -
13 |   |   |   |   |   |   | - - - |   | - - -
14 |   |   |   |   |   |   | - - - |   | - - -
15 |   |   |   |   |   |   | - - - |   | - - -
16 |   |   |   |   |   |   | - - - |   | - - -
17 |   |   |   |   |   |   | - - - |   | - - -
18 |   |   |   |   |   |   | - - - |   | - - -
19 .....d b..... Astra Move
20 """
21
22
23 def classificar_rover(bateria_pct):
24     bateria_minima = limites_seguranca[2]
25     if bateria_pct >= bateria_minima:
26         return "Rover seguro para operação"
27     elif bateria_pct >= 50.0:
28         return "ATENÇÃO. Rover em condição de ALERTA"
29     else:
30         return "ALERTA!!! Rover em condição CRÍTICA!"
31
32
33 def classificar_modulo(modulo_id):
34     co2_limite = limites_seguranca[0]
35     o2_limite = limites_seguranca[1]
36
37     leituras_modulo = [leitura for leitura in leituras_ambientais if leitura["modulo"] == modulo_id]
38     if not leituras_modulo:
39         return "Nenhuma leitura disponível para o módulo especificado."
40
41     ultima_leitura = leituras_modulo[-1]
42
43     if ultima_leitura["co2_ppm"] > co2_limite or ultima_leitura["o2_pct"] < o2_limite:
44         return f"ALERTA!!! (CO2: {ultima_leitura['co2_ppm']} ppm, O2: {ultima_leitura['o2_pct']}%) - Condição CRÍTICA!"
45     else:
46         return f"Módulo seguro para operação (CO2: {ultima_leitura['co2_ppm']} ppm, O2: {ultima_leitura['o2_pct']}%)"
47
48
49 def calcular_indicadores_globais():
50     total_capacidade = sum(modulo["capacidade"] for modulo in modulos)
51     total_ocupacao = sum(modulo["ocupacao"] for modulo in modulos)
52     taxa_ocupacao = (total_ocupacao / total_capacidade) * 100 if total_capacidade > 0 else 0

```

```

48
49 def calcular_indicadores_globais():
50     total_capacidade = sum(modulo["capacidade"] for modulo in modulos)
51     total_ocupacao = sum(modulo["ocupacao"] for modulo in modulos)
52     taxa_ocupacao = (total_ocupacao / total_capacidade) * 100 if total_capacidade > 0 else 0
53
54     total_autonomia = sum(veiculo["autonomia_km"] for veiculo in veiculos)
55     media_autonomia = total_autonomia / len(veiculos) if veiculos else 0
56
57     vagas_totais = sum(modulo["capacidade"] - modulo["ocupacao"] for modulo in modulos)
58
59     return taxa_ocupacao, media_autonomia, vagas_totais
60
61
62 def analisar_matriz_terreno(matriz):
63     seguro = 0
64     atencao = 0
65     perigoso = 0
66
67     for linha in matriz:
68         for celula in linha:
69             if celula == 1:
70                 seguro += 1
71             elif celula == 2:
72                 atencao += 1
73             elif celula == 0:
74                 perigoso += 1
75     return seguro, atencao, perigoso
76
77
78 def executar_diagnostico():
79     print("=====")
80     print("          CENTRO DE CONTROLE LUNAR | DIAGNÓSTICO OPERACIONAL          ")
81     print("=====")
82
83     print("Status dos módulos de residência e habitat")
84     modulos_alerta = 0
85     for modulo in modulos:
86         vagas = modulo["capacidade"] - modulo["ocupacao"]
87         status_ambiental = classificar_modulo(modulo["id"])
88
89         if "CRÍTICA" in status_ambiental:
90             modulos_alerta += 1
91             print(f"ALERTA!!! Módulo {modulo['nome']} em condição CRÍTICA! (Vagas: {vagas})")
92     print("\n" + "-" * 66 + "\n")
93
94     print("Status dos rovers e veículos de transporte")
95     rovers_aptos = 0
96     for veiculo in veiculos:
97         status_rover = classificar_rover(veiculo["bateria_pct"])
98         if "seguro" in status_rover:
99             rovers_aptos += 1
100         print(f"Rover {veiculo['modelo']} (Bateria: {veiculo['bateria_pct']}%) - {status_rover}")
101     print("\n" + "-" * 66 + "\n")

```

```

102     seg, atenc, per = analisar_matriz_terreno(mapa_terreno)
103     print("Mapeamento do terreno ao redor da base")
104     print(f"Seguros: {seg}, Atenção: {atenc}, Perigosos: {per}")
105     print("\n" + "-" * 66 + "\n")
106
107
108     taxa_ocupacao, media_autonomia, vagas_totais = calcular_indicadores_globais()
109     print("Indicadores globais da base lunar")
110     print(f"Taxa de ocupação da base: {taxa_ocupacao:.1f}%")
111     print(f"Média de autonomia dos rovers: {media_autonomia:.2f} km")
112     print(f"Vagas totais disponíveis nos módulos: {vagas_totais}")
113     print("\n" + "-" * 66 + "\n")
114
115     print("Diagnóstico completo. Encerrando o relatório do Centro de Controle Lunar.")
116     if modulos_alerta > 0:
117         print(f"ALERTA!!! Existem {modulos_alerta} módulos em condição CRÍTICA. Ação imediata recomendada.")
118         print(f"Existem {vagas_totais} vagas disponíveis e {rovers_aptos} rovers APTOS para realocação/resgate.")
119         print("Recomendação: Execute o 'Programa 02' para planejar a evacuação dos módulos afetados.")
120     else:
121         print("Todos os módulos estão operando dentro dos parâmetros de segurança. Nenhuma ação corretiva necessária.")
122
123     print(ARTE_FINAL)
124
125
126 if __name__ == "__main__":
127     executar_diagnostico()

```

No terminal:

Programa 2:

```
LU.N.A.R_programa2.py > ...
1  from mobilidadekit_dados import veiculos, rotas, cargas, pontos_interesse, mapa_terreno
2  from residenciakit_dados import modulos, leituras_ambientais, mapa_base
3
4  condicoes_missao = ("Capacidade", "Autonomia", "Risco")
5  condicao1, condicao2, condicao3 = condicoes_missao
6
7  def mostrar_veiculos():
8      print("\n===== ROVERS =====")
9
10     for i, veiculo in enumerate(veiculos):
11         print(
12             i + 1,
13             "-",
14             veiculo["id"],
15             "|",
16             veiculo["modelo"],
17             "| Capacidade:",
18             veiculo["capacidade_kg"],
19             "kg | Autonomia:",
20             veiculo["autonomia_km"],
21             "km"
22         )
23
24
25
26
27  def mostrar_cargas():
28      print("\n===== CARGAS =====")
29
30     for i, carga in enumerate(cargas):
31         print(
32             i + 1,
33             "-",
34             carga["id"],
35             "|",
36             carga["tipo"],
37             "|",
38             carga["massa_kg"],
39             "kg"
40         )
41
```

```

26 def mostrar_cargas():
27     print("\n===== CARGAS =====")
28
29     for i, carga in enumerate(cargas):
30         print(
31             i + 1,
32             "-",
33             carga["id"],
34             "|",
35             carga["tipo"],
36             "|",
37             carga["massa_kg"],
38             "kg"
39         )
40
41
42
43
44 def mostrar_rotas():
45     print("\n===== ROTAS =====")
46
47     for i, rota in enumerate(rotas):
48         print(
49             i + 1,
50             "-",
51             rota["id"],
52             "|",
53             rota["origem"],
54             "->",
55             rota["destino"],
56             "|",
57             rota["distancia_km"],
58             "km | Risco:",
59             rota["risco"]
60         )
61
62
63
64
65 print("=" * 50)
66 print("    SIMULADOR DE OPERAÇÃO LUNAR")
67 print("=" * 50)
68
69
70
71
72 mostrar_veiculos()
73
74 opcao_rover = int(input("\nEscolha o rover: "))
75
76 rover = veiculos[opcao_rover - 1]

```



```

80
81 mostrar_cargas()
82
83 opcao_carga = int(input("\nEscolha a carga: "))
84
85 carga = cargas[opcao_carga - 1]
86
87
88
89
90 mostrar_rotas()
91
92 opcao_rota = int(input("\nEscolha a rota: "))
93
94 rota = rotas[opcao_rota - 1]
95
96
97
98
99 distancia_total = rota["distancia_km"] * 2
100
101 tempo = distancia_total / rover["velocidade_kmh"]
102
103
104
105
106 capacidade_ok = carga["massa_kg"] <= rover["capacidade_kg"]
107
108 autonomia_ok = distancia_total <= rover["autonomia_km"]
109
110
111
112
113 if capacidade_ok and autonomia_ok:
114
115     if rota["risco"] == "alto":
116         resultado = "MISSÃO AUTORIZADA COM RESTRIÇÕES"
117         recomendacao = "A missão é possível, mas a rota possui risco alto."
118         condicoes_missao = condicao3
119
120     elif rota["risco"] == "medio":
121         resultado = "MISSÃO AUTORIZADA COM RESTRIÇÕES"
122         recomendacao = "A missão é possível, mas exige atenção à rota."
123         condicoes_missao = condicao1
124
125     else:
126         resultado = "MISSÃO AUTORIZADA"
127         recomendacao = "O rover possui capacidade e autonomia suficientes."
128         condicoes_missao = condicao2
129 else:
130
131     resultado = "MISSÃO NEGADA"

```

```

129
130     resultado = "MISSÃO NEGADA"
131
132     if not capacidade_ok:
133         recomendacao = "A carga ultrapassa a capacidade do rover."
134         condicoes_missao = condicao3
135
136     else:
137         recomendacao = "A autonomia do rover não é suficiente."
138         condicoes_missao = condicao3
139
140
141
142
143     print("\n")
144     print("=" * 50)
145     print("          RESULTADO DA SIMULAÇÃO")
146     print("=" * 50)
147
148     print("Rover:", rover["id"], "-", rover["modelo"])
149     print("Carga:", carga["tipo"])
150     print("Massa da carga:", carga["massa_kg"], "kg")
151     print("Capacidade do rover:", rover["capacidade_kg"], "kg")
152
153     print("\nRota:", rota["id"])
154     print("Destino:", rota["destino"])
155     print("Distância de ida:", rota["distancia_km"], "km")
156     print("Distância total:", distancia_total, "km")
157
158     print("Autonomia:", rover["autonomia_km"], "km")
159     print("Bateria:", rover["bateria_pct"], "%")
160     print("Risco:", rota["risco"])
161
162     print("Tempo estimado:", round(tempo, 2), "horas")
163
164
165
166
167     print("\nCoordenada da Base Alpha:",
168           pontos_interesse[0]["coordenada"])
169
170
171
172
173     print("Posição inicial do mapa:",
174           mapa_terreno[0][0])
175
176
177     print("\nRESULTADO:", resultado)
178     print("RECOMENDAÇÃO:", recomendacao)
179     print("CONDIÇÕES AVALIADAS:", condicoes_missao)
180     print("=" * 50)

```

No terminal:

=====

SIMULADOR DE OPERAÇÃO LUNAR

=====

===== ROVERS =====

- 1 - RV-01 | Scout | Capacidade: 180 kg | Autonomia: 22.0 km
- 2 - RV-02 | Cargo X1 | Capacidade: 480 kg | Autonomia: 15.0 km
- 3 - RV-03 | Cargo X2 | Capacidade: 650 kg | Autonomia: 11.5 km
- 4 - RV-04 | Explorer | Capacidade: 120 kg | Autonomia: 28.0 km
- 5 - RV-05 | Crew Rover | Capacidade: 300 kg | Autonomia: 18.5 km
- 6 - RV-06 | Utility | Capacidade: 360 kg | Autonomia: 16.0 km

Escolha o rover: 3

===== CARGAS =====

- 1 - CG-01 | regolito | 120 kg
- 2 - CG-02 | gelo extraído | 260 kg
- 3 - CG-03 | baterias | 180 kg
- 4 - CG-04 | peças mecanicas | 310 kg
- 5 - CG-05 | alimentos | 95 kg
- 6 - CG-06 | equipamentos científicos | 220 kg

Escolha a carga: 2

===== ROTAS =====

- 1 - RT-01 | Base Alpha -> Mina A | 7.2 km | Risco: baixo
- 2 - RT-02 | Base Alpha -> Mina B | 10.8 km | Risco: medio
- 3 - RT-03 | Mina A -> Deposito | 5.4 km | Risco: baixo
- 4 - RT-04 | Deposito -> Base Alpha | 6.1 km | Risco: alto
- 5 - RT-05 | Base Alpha -> Cratera Sul | 13.5 km | Risco: medio
- 6 - RT-06 | Mina B -> Deposito | 8.6 km | Risco: alto

Escolha a rota: 6

=====

RESULTADO DA SIMULAÇÃO

=====

Rover: RV-03 - Cargo X2
Carga: gelo extraído
Massa da carga: 260 kg
Capacidade do rover: 650 kg

Rota: RT-06
Destino: Deposito
Distância de ida: 8.6 km
Distância total: 17.2 km
Autonomia: 11.5 km
Bateria: 78 %
Risco: alto
Tempo estimado: 3.44 horas

Coordenada da Base Alpha: (10, 12)
Posição inicial do mapa: 1

RESULTADO: MISSÃO NEGADA
RECOMENDAÇÃO: A autonomia do rover não é suficiente.
CONDIÇÕES AVALIADAS: Risco

=====